

UNIVERSITÀ DEGLI STUDI DI SALERNO
Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica Applicata
Ingegneria Informatica
Corso di Programmazione Java Avanzata



Relazione finale di progetto

GuessTheWord G1

Candidati: Carmine Muollo (0612709530)
Sabrina Soriano (0612709851)
William Menza (0612709893)
Davide Andrea Odierna (0612710197)

Anno Accademico: 2025 - 2026

Indice

1	Introduzione al problema e soluzione proposta	1
1.1	Presentazione del problema e degli obiettivi	1
1.2	La soluzione proposta e contesto di sistema	1
1.3	Suddivisione dei ruoli e contributi del team	2
1.4	Tecnologie utilizzate	2
2	Progettazione del sistema	3
2.1	Requisiti e casi d'uso	3
2.1.1	Requisiti non funzionali	4
2.1.2	Casi d'uso	4
2.2	Architettura del sistema	5
2.3	Modello dei dati e persistenza	6
2.3.1	Diagramma Entità-Relazione (E-R)	6
2.3.2	Schema Relazionale	6
2.4	Protocollo di comunicazione	6
2.5	Diagramma di sequenza del gioco	7
2.6	Diagrammi delle classi	9
2.6.1	Diagramma delle classi del package network nel progetto Client	9
2.6.2	Diagramma delle classi del package network nel progetto Server	10
2.6.3	Diagramma delle classi della logica di gioco	11
2.6.4	Diagramma delle classi della persistenza	12
2.6.5	Diagramma delle classi della logica di analisi della term frequency	13
2.7	Scelte di progettazione e design pattern	13
3	Implementazione	15
3.1	Interfaccia grafica del sistema	15
3.2	Frammenti di codice rilevanti	20
3.2.1	Analisi della frequenza dei termini	20
3.2.2	Cifrario di Cesare e generazione dello shift	21
3.2.3	Protocollo di comunicazione	22
3.2.4	Gestione della connessione socket	22
3.2.5	Persistenza tramite pattern DAO	23
3.2.6	Inizializzazione dello schema e gestione delle connessioni	24
3.2.7	Caching serializzato dell'analisi delle parole	25
4	Conclusioni e sviluppi futuri	26
4.1	Risultati conseguiti	26
4.2	Limitazioni riscontrate	26
4.3	Sviluppi futuri	26

Capitolo 1

Introduzione al problema e soluzione proposta

1.1 Presentazione del problema e degli obiettivi

La progettazione di sistemi distribuiti destinati all'intrattenimento interattivo in tempo reale pone sfide rilevanti in termini di sincronizzazione dello stato, efficienza dei protocolli di comunicazione ed usabilità delle interfacce grafiche.

Il progetto GuessTheWord si colloca in questo scenario proponendosi come un sistema software client-server finalizzato alla gestione di partite competitive tra utenti connessi in rete. L'obiettivo fondamentale del sistema consiste nel consentire a due giocatori di sfidarsi in tempo reale nel decifrare una parola segreta, all'interno di un estratto di un testo letterario ed ofuscata tramite l'applicazione del cifrario di Cesare. La complessità del sistema risiede nella necessità di elaborare dinamicamente i documenti di testo per l'estrazione delle parole, applicare algoritmi di cifratura coerenti con la difficoltà selezionata, gestire una coda di accoppiamento per i giocatori e garantire la persistenza dei dati relativi ad utenti, sfide e partite giocate, preservando la reattività delle interfacce grafiche sia sul lato client che su quello amministrativo lato server.

1.2 La soluzione proposta e contesto di sistema

La soluzione da noi progettata ed implementata si articola su un'architettura client-server disaccoppiata basata su socket di rete TCP. Il server agisce da coordinatore centrale ed esegue i compiti di autenticazione degli utenti, ricezione dei documenti di testo da parte dell'amministratore, analisi statistica della term frequency per la generazione delle sfide, gestione delle lobby e sincronizzazione delle sessioni di gioco attive. Il server integra inoltre una base di dati relazionale locale per garantire l'immutabilità dello storico delle partite e la tracciabilità delle sfide vinte dagli utenti.

Il client si configura come un'applicazione JavaFX che si connette al server per consentire all'utente di registrarsi, effettuare il login, selezionare la difficoltà di gioco desiderata, attendere l'accoppiamento con un avversario in una lobby di attesa virtuale e prendere parte alla sfida inviando i propri tentativi di risoluzione (massimo tre) entro un limite temporale stabilito (sessanta secondi). Il sistema gestisce le disconnessioni improvvise del server, i timeout e la visualizzazione dello storico delle partite in modo da preservare la coerenza dello stato globale del gioco.

1.3 Suddivisione dei ruoli e contributi del team

Lo sviluppo del sistema è stato condotto secondo criteri di ingegneria del software basati sulla separazione delle responsabilità, riflettendo le competenze individuali all'interno della struttura dei package e delle classi:

- *Carminé Muollo*: progettazione e implementazione del livello di persistenza dei dati, definendo la classe per la gestione della connessione al database ed i relativi oggetti di accesso ai dati per le entità di dominio (DAO); sviluppo dell'interfaccia amministrativa del server e la logica di serializzazione per la memorizzazione persistente della cache dei risultati dell'analisi.
- *William Menza*: realizzazione dell'intero front-end del client di gioco, implementazione dei controller delle view per il login, la selezione della difficoltà, la lobby di attesa, la schermata di gioco ed il visualizzatore dello storico personale, sviluppo dei service lato client per l'interazione con la connessione di rete e le funzioni di utilità per la validazione degli input e la codifica crittografica delle credenziali (password).
- *Sabrina Soriano*: progettazione dell'infrastruttura di rete del server, realizzando il socket server multithreaded, la logica di gestione delle connessioni client individuali, il registro centralizzato per il monitoraggio dei client attivi, il protocollo di messaggistica di rete ed il task di ascolto asincrono impiegato dal client per ricevere eventi in background.
- *Davide Andrea Odierna*: sviluppo della logica centrale del motore di gioco, definendo il gestore delle sessioni, la logica di accoppiamento degli utenti in base alle preferenze di difficoltà ed il sistema di sincronizzazione dei thread all'interno delle sessioni per impedire race condition durante la sottomissione delle risposte concorrenti e il servizio asincrono per l'analisi dei testi basato sulla concorrenza in JavaFX.

1.4 Tecnologie utilizzate

Per lo sviluppo dell'intero ecosistema GuessTheWord abbiamo adottato le seguenti tecnologie e linguaggi:

- *Java Development Kit 8*: come linguaggio di programmazione di riferimento, sfruttando le espressioni lambda, la Stream API per il calcolo delle frequenze testuali e la gestione avanzata dei thread.
- *JavaFX*: per la costruzione delle interfacce grafiche desktop sia sul client che sul pannello di amministrazione del server.
- *SQLite*: come motore di database relazionale leggero ed embedded, integrato direttamente nell'applicazione server senza la necessità di un DBMS esterno dedicato.
- *JDBC (Java Database Connectivity)*: per l'esecuzione di interrogazioni SQL strutturate, l'apertura delle connessioni e la gestione transazionale dei dati.
- *Java Object Serialization*: per il salvataggio su file della cache dei risultati dell'analisi testuale.
- *Cascading Style Sheets (CSS)*: per l'applicazione di stili personalizzati all'interfaccia grafica.

Capitolo 2

Progettazione del sistema

2.1 Requisiti e casi d'uso

La fase di analisi del sistema ha condotto alla definizione dei requisiti funzionali per le due tipologie di attori che interagiscono con l'applicazione: l'amministratore del server ed il giocatore. Di seguito viene riportata la tabella riassuntiva dei requisiti identificati.

ID Requisito	Descrizione
RF-01	Autenticazione al pannello di controllo del server per l'amministratore.
RF-02	Selezione di uno o più documenti in formato testuale (.txt) da parte dell'amministratore per l'analisi.
RF-03	Elaborazione asincrona del testo per estrarre le parole chiave significative per le sfide.
RF-04	Salvataggio in cache dei risultati dell'analisi tramite serializzazione (.ser).
RF-05	Caricamento da cache dei risultati dell'analisi precedentemente memorizzati.
RF-06	Visualizzazione delle statistiche globali degli utenti e dello storico delle partite sul server.
RF-07	Registrazione di un nuovo profilo utente giocatore con validazione dei campi lato client.
RF-08	Autenticazione degli utenti tramite username e password.
RF-09	Selezione della difficoltà della partita da disputare (facile, media, difficile).
RF-10	Inserimento in una lobby di attesa e abbinamento automatico con un avversario in una sfida di pari difficoltà.
RF-11	Partecipazione ad una partita in tempo reale con visualizzazione dell'estratto di testo contenente la parola cifrata.
RF-12	Visualizzazione dello storico personale delle partite giocate dal singolo utente.

Tabella 2.1: Requisiti funzionali del sistema

2.1.1 Requisiti non funzionali

Al fine di garantire l'affidabilità, la sicurezza e la corretta esecuzione del sistema, sono stati definiti i requisiti non funzionali riportati nella tabella seguente.

ID Requisito	Descrizione
RNF-01	Compatibilità con JDK 8.
RNF-02	Reattività della GUI, con tutte le operazioni di lettura/scrittura di rete e interrogazioni del database eseguite al di fuori del thread dell'applicazione JavaFX.
RNF-03	Utilizzo esclusivo di percorsi relativi per: il database SQLite, file di proprietà, immagini, classi di stile e file di testo.
RNF-04	Sicurezza dei dati, con password mai memorizzate in chiaro e applicazione di un algoritmo di hashing sicuro prima del salvataggio.
RNF-05	Packaging autonomo, con la generazione di due file JAR eseguibili (server.jar e client.jar) che includono tutte le dipendenze, un file readme.txt contenente le credenziali dell'amministratore e di due utenti di test, e file .properties per la configurazione del server e del client.

Tabella 2.2: Requisiti non funzionali del sistema

2.1.2 Casi d'uso

Per delineare compiutamente le interazioni tra gli attori e il sistema, abbiamo definito i casi d'uso principali del sistema GuessTheWord, riportati nella tabella seguente:

ID Caso d'Uso	Attore	Descrizione
UC-01	Giocatore	Il giocatore inserisce credenziali valide e il sistema concede l'accesso alla lobby di selezione della difficoltà.
UC-02	Giocatore	Il giocatore si registra compilando i campi username e password e il sistema memorizza il profilo nel database relazionale.
UC-03	Giocatore	Il giocatore sceglie la difficoltà e il sistema lo colloca nella schermata di caricamento, in attesa di un avversario.
UC-04	Giocatore	Il giocatore viene abbinato ad un avversario e il sistema avvia la partita inviando la parola cifrata (del caso in cui non venga caricato alcun file dall'amministratore, quest'ultima viene selezionata randomicamente da un elenco di parole predefinito) o l'estratto di testo contenente la parola cifrata.
UC-05	Giocatore	Il giocatore invia la risposta e il sistema ne verifica l'esattezza aggiornando lo stato della sessione.
UC-06	Giocatore	Il giocatore richiede lo storico e il sistema recupera e visualizza le partite precedentemente giocate.
UC-07	Amministratore	L'amministratore inserisce le credenziali e il sistema garantisce l'accesso al pannello di controllo.
UC-08	Amministratore	L'amministratore seleziona file di testo e il sistema estrae in background la parola chiave.
UC-09	Amministratore	L'amministratore salva i risultati dell'analisi e, se richiesto dall'amministratore, il sistema serializza l'oggetto (.ser), salvandolo sul disco.
UC-10	Amministratore	L'amministratore consulta la classifica globale e il sistema mostra le statistiche degli utenti che hanno vinto almeno una partita (username, partite vinte, tempo medio di risposta).

Tabella 2.3: Casi d'uso del sistema

2.2 Architettura del sistema

Il sistema si basa su uno stile architetturale client-server centralizzato. La comunicazione di rete avviene tramite canali socket persistenti operanti sul protocollo di trasporto TCP, scelta che garantisce l'affidabilità nella consegna dei messaggi e l'ordine dei pacchetti inviati. Il server implementa un modello a thread concorrenti per supportare connessioni multiple simultanee. All'avvio, la classe dedicata alla gestione del server si pone in ascolto sulla porta 5000, come definito nel file di configurazione *server.properties*. Per ogni connessione accettata, viene generato un thread indipendente associato ad un gestore del client. Questo componente incapsula i flussi di input e output della socket ed esegue un ciclo continuo di lettura dei messaggi di rete, delegando l'elaborazione dei comandi alla logica di business dell'applicazione e garantendo che il server rimanga in attesa di nuove connessioni sul thread principale.

2.3 Modello dei dati e persistenza

Il livello di persistenza è strutturato per archiviare i dati di gioco all'interno di un database relazionale gestito con SQLite. La struttura delle tabelle ed i vincoli di integrità referenziale sono basati sul diagramma entity relationship.

2.3.1 Diagramma Entità-Relazione (E-R)

Di seguito viene mostrato lo schema E-R del database mediante diagramma grafico.

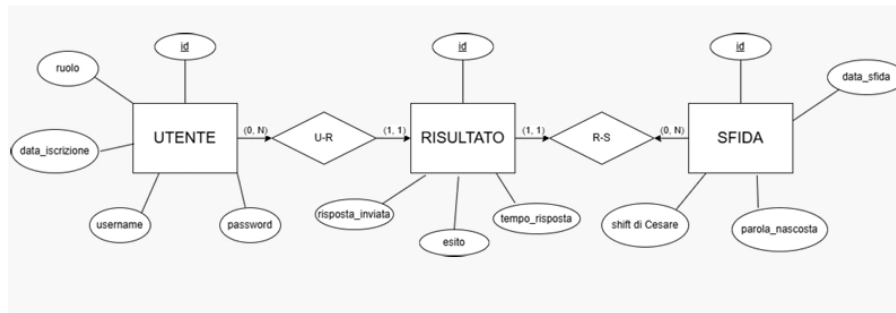


Figura 2.1: Schema E-R

2.3.2 Schema Relazionale

Il modello dei dati si basa sulle tabelle definite all'interno della classe *DatabaseManager*:

- *utenti*: memorizza l'anagrafica, le credenziali e il ruolo di ciascun utente.
- *sfide*: tiene traccia delle parole nascoste, dello shift del cifrario applicato, della data di creazione e della difficoltà associata.
- *risultati*: rappresenta la tabella di associazione per tenere traccia dell'esito delle singole partite per ciascun partecipante.

Le relazioni logiche sono definite come segue:

- `utenti (id, username, password, ruolo, data_iscrizione)`
Primary Key: `id`, UNIQUE: `username`.
- `sfide (id, parola_nascosta, shift_cesare, data_sfida, difficulta)`
Primary Key: `id`.
- `risultati (id, id_utente, id_sfida, esito, risposta_inviata, tempo_risposta)`
Primary Key: `id`, Foreign Keys: `id_utente` riferenzia `utenti(id)`, `id_sfida` riferenzia `sfide(id)` con politiche di cancellazione in cascata.

2.4 Protocollo di comunicazione

La comunicazione tra client e server è regolata da un protocollo testuale personalizzato. I messaggi sono formattati utilizzando un delimitatore specifico per isolare il comando dai suoi parametri. La scelta del delimitatore è ricaduta sul carattere di controllo non stampabile Unit Separator (`\u001F`), prevenendo conflitti con i caratteri speciali (ad esempio :) che potrebbero essere inseriti nei campi di input degli utenti o estratti dai documenti analizzati. La struttura generale di un messaggio si presenta nella forma: `COMANDO\u001Fparametro1\u001Fparametro2`.

Comando	Mittente	Parametri	Descrizione
AUTH_LOGIN	Client	username, password	Richiesta di login dell'utente.
AUTH_REGISTER	Client	username, password	Richiesta di registrazione di un nuovo profilo utente.
AUTH_OK	Server	username	Notifica di avvenuta autenticazione con successo.
AUTH_FAIL	Server	motivo	Notifica di fallimento dell'autenticazione o errore generico.
WAITING	Client / Server	nessuno	Inserimento nella lobby di attesa o notifica dello stato di attesa.
OPPONENT_FOUND	Server	nessuno	Notifica dell'avvenuto accoppiamento con l'avversario.
GAME_START	Server	testoCifrato, shift, durata	Avvio del round di gioco con l'invio dei dati della sfida.
GAME_ANSWER	Client	parolaProposta	Invio del tentativo di risposta.
GAME_WIN	Server	tempoRisposta, parola	Notifica di vittoria con relativi dati di riepilogo della sfida.
GAME_LOSE	Server	vincitore, tempo, parola	Notifica di sconfitta con dettagli sul vincitore e tempo di sottomissione.
GAME_TIMEOUT	Server	parolaNascosta	Fine del round dovuto allo scadere del tempo o all'esaurimento dei tentativi.
REQ_HISTORY	Client	nessuno	Richiesta dello storico personale dei risultati delle sfide.
HISTORY_DATA	Server	datiStorico	Invio dei dati dello storico personale separati per partita.
SERVER_SHUTDOWN	Server	nessuno	Notifica ai client connessi dell'arresto imminente del server.

Tabella 2.4: Protocollo di comunicazione GuessTheWord

2.5 Diagramma di sequenza del gioco

il diagramma di sequenza descrive la fase dall'apertura della connessione al salvataggio del risultato, in particolare:

- *GameServer.accept()*: crea due ClientHandler distinti (uno per giocatore) e li sottomette al thread pool (Connessione).
- *Autenticazione*: Ciascun ClientHandler chiama UserDAO.authenticate().
- *GameManager.addToLobby()*: il primo giocatore resta in coda, il secondo fa scattare l'abbinamento con ChallengePreparator.prepare() e la creazione della GameSession (Matching).

- *GameSession.start()*: cifra la parola e invia GAME_START; la risposta del giocatore passa da ClientHandler.handleAnswer() a GameSession.handleAnswer()
- *Ramo alt*: per risposta corretta (salvataggio via MatchPersistenceService, notifica GAME_WIN/GAM_LOSE, rimozione sessione) oppure risposta errata (Esito partita)

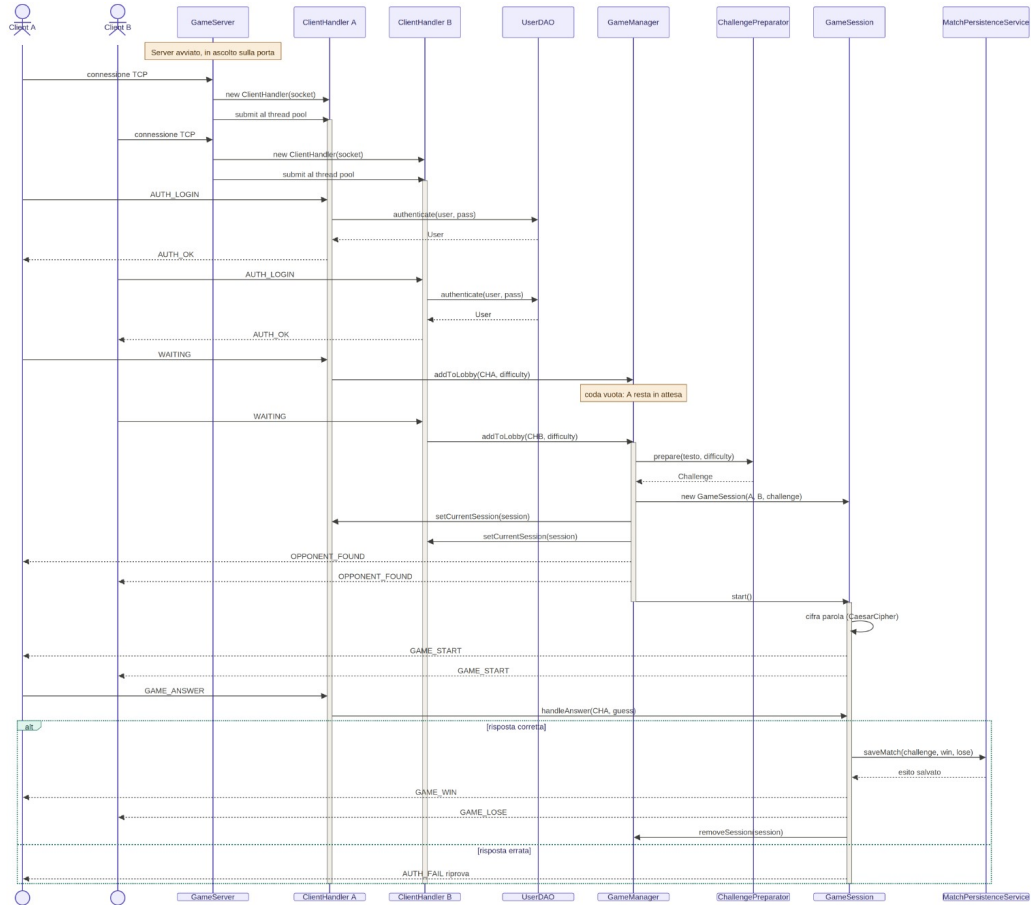


Figura 2.2: Sequence diagram

2.6 Diagrammi delle classi

2.6.1 Diagramma delle classi del package network nel progetto Client

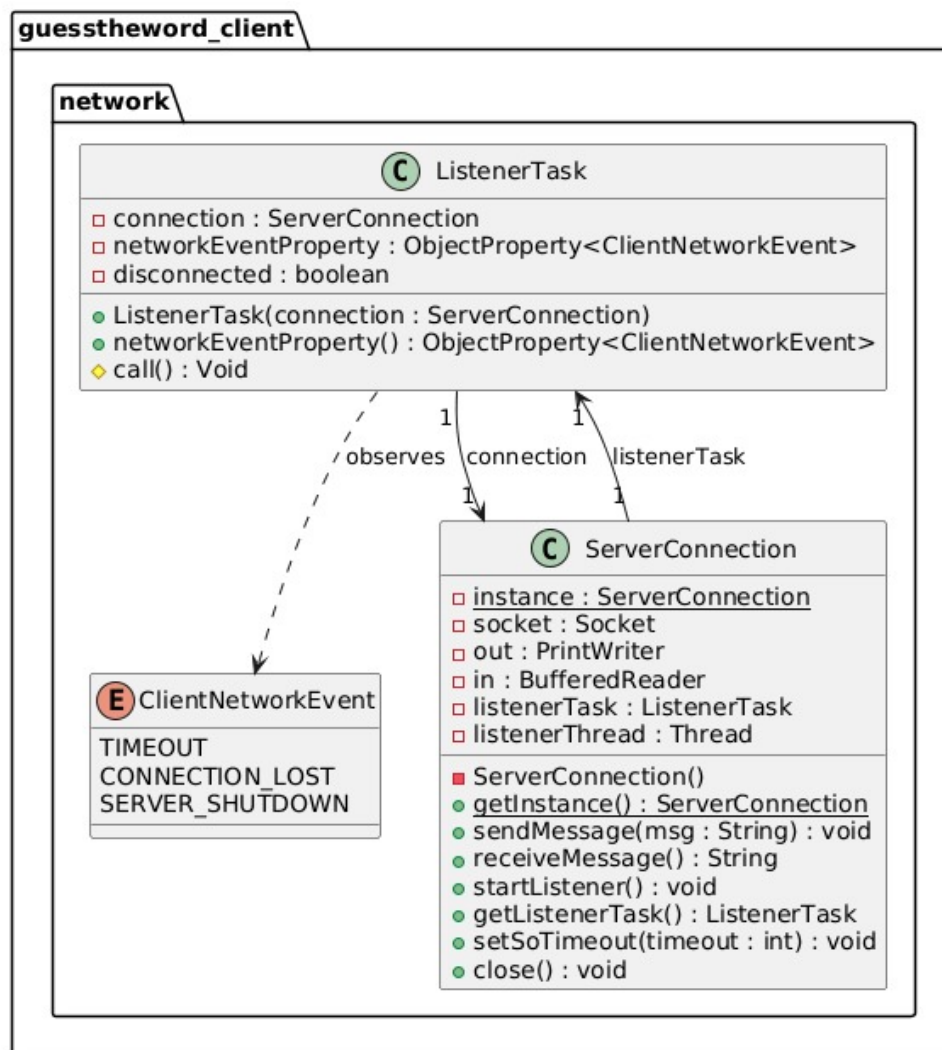


Figura 2.3: Diagramma delle classi del package network

Il diagramma illustra l'architettura del livello di rete lato Client, responsabile della comunicazione socket con il Server. La classe centrale è **ServerConnection**, implementata tramite il pattern Singleton per garantire che l'intera applicazione utilizzi un'unica connessione condivisa per l'invio e la ricezione dei messaggi. Per gestire i messaggi in arrivo in modo asincrono.

ServerConnection istanzia un **ListenerTask**; cioè un thread in background che ascolta continuamente lo stream di input senza bloccare l'interfaccia grafica (JavaFX Application Thread). Il task è strettamente accoppiato all'enumerazione **ClientNetworkEvent**, utilizzata per notificare alla GUI (tramite l'aggiornamento di Properties reattive) eventuali anomalie di rete, come timeout, caduta di connessione o spegnimento improvviso del server, permettendo all'applicazione di reagire prontamente ai casi di errore o a situazioni impreviste.

2.6.2 Diagramma delle classi del package network nel progetto Server

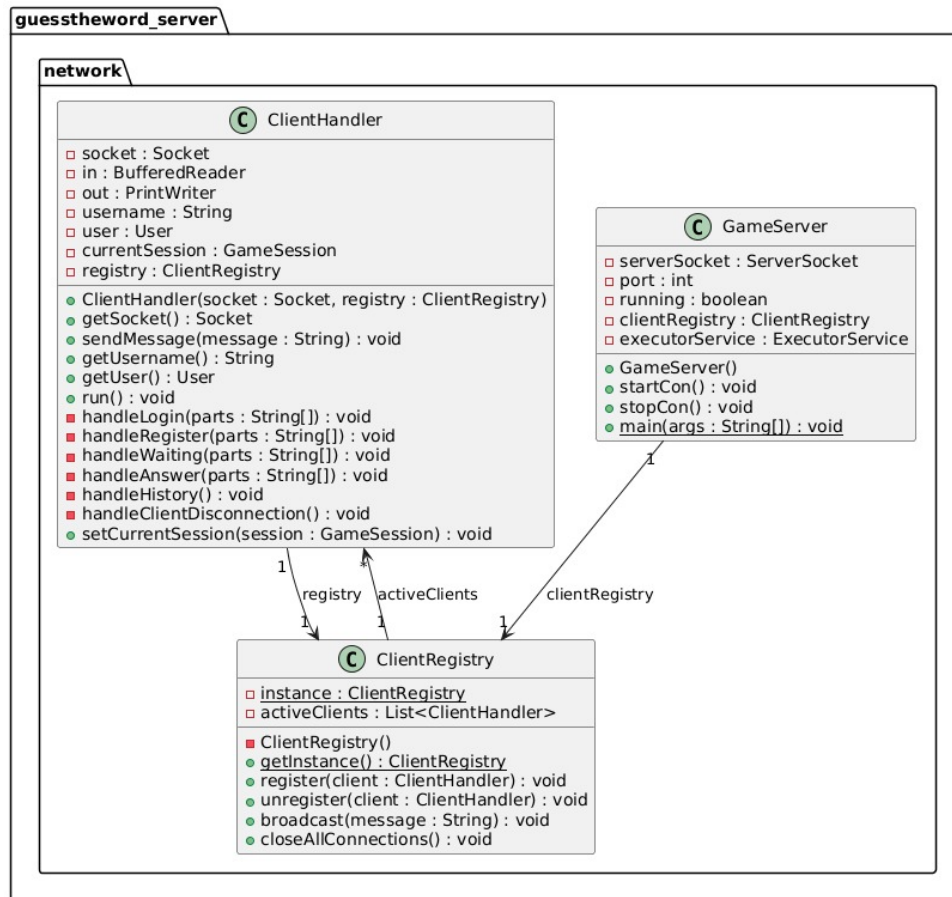


Figura 2.4: Diagramma delle classi del package network

Questo diagramma descrive il nucleo dell'infrastruttura di rete lato Server, dedicato alla gestione concorrente dei giocatori. La classe `GameServer` rappresenta l'entry point del server, incaricata di mettersi in ascolto sulla porta designata e di accettare le nuove connessioni tramite un pool di thread (`ExecutorService`). Ogni volta che un client si connette, viene istanziato un `ClientHandler` (che funge da `Runnable`). Questa classe si occupa di servire in via esclusiva le richieste di quello specifico giocatore. La gestione e il monitoraggio dei client attivi sono delegati a `ClientRegistry`, un Singleton che mantiene una lista aggiornata di tutti i `ClientHandler` connessi e fornisce funzionalità di notifica broadcast. Quando un client si disconnette, il suo `ClientHandler` invoca il registry per deregistrarsi, garantendo un ciclo di vita pulito e sicuro delle connessioni.

2.6.3 Diagramma delle classi della logica di gioco

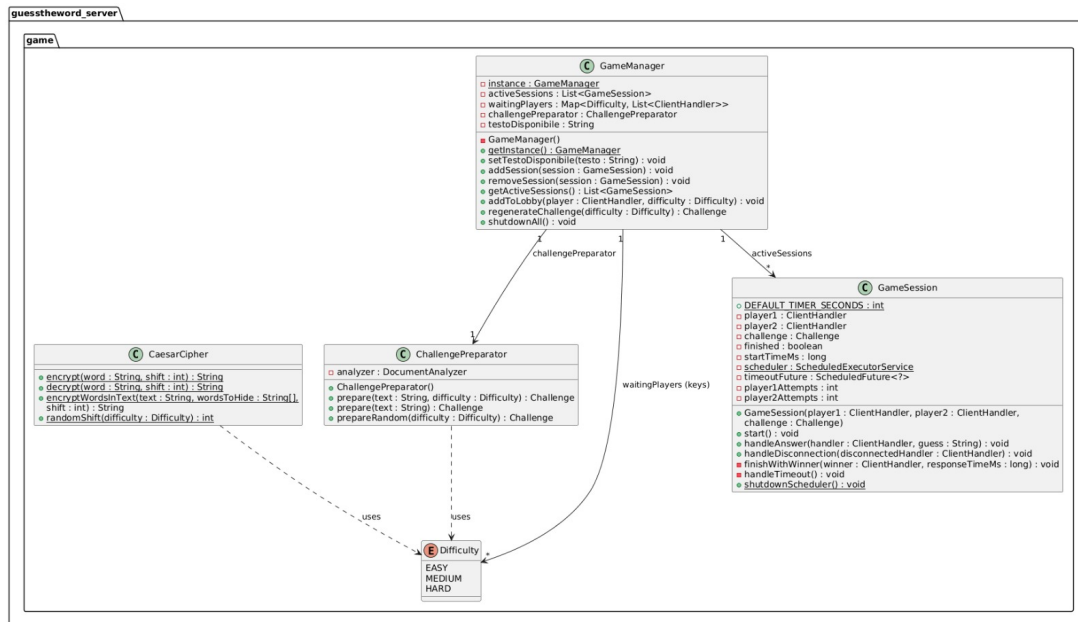


Figura 2.5: Diagramma delle classi del package game del progetto Server

Il diagramma focalizza l'attenzione sulla logica di dominio (business logic) del Server che orchestra il matchmaking e le partite. Il coordinatore principale è `GameManager` (pattern Singleton), che detiene le code di attesa dei giocatori (una mappa `waitingPlayers` divisa per livello di `Difficulty`) e tiene traccia di tutte le partite in corso (`activeSessions`). Quando due giocatori vengono abbinati, `GameManager` si avvale della classe `ChallengePreparator` per generare una nuova sfida (la parola segreta e il testo cifrato), la quale a sua volta sfrutta la classe `CaesarCipher` per crittografare la parola in base alla difficoltà. Viene quindi istanziata una `GameSession` che racchiude lo stato della partita corrente tra i due `ClientHandler`, gestendo un timer asincrono condiviso (tramite `ScheduledExecutorService`), il conteggio dei tentativi e il controllo incrociato delle risposte.

2.6.4 Diagramma delle classi della persistenza

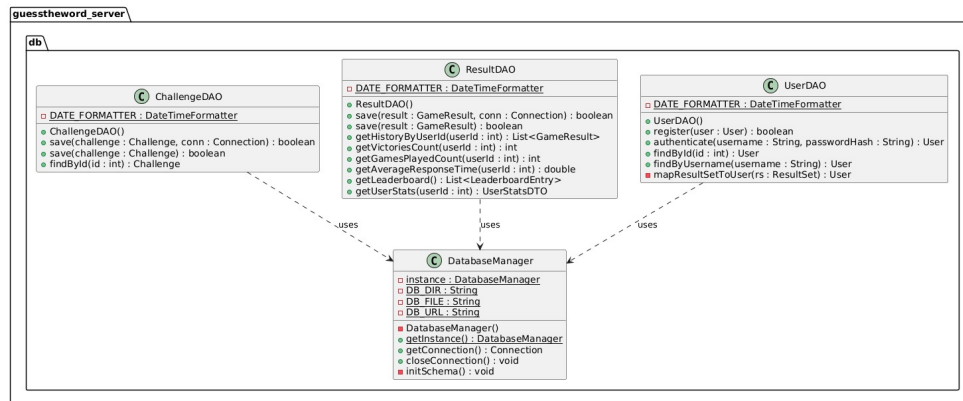


Figura 2.6: Diagramma delle classi del package db

Questo schema illustra il livello di del Data Access Layer (persistenza dei dati) sul Server, strutturato elegantemente tramite il Pattern DAO (Data Access Object). L'architettura è suddivisa in tre classi DAO specifiche, ognuna responsabile di una specifica entità: UserDAO (gestione delle credenziali e registrazione utenti), ResultDAO (statistiche di gioco, storico partite e generazione della leaderboard), e ChallengeDAO (salvataggio dei meta-dati delle singole sfide). Tutti e tre i DAO condividono una forte dipendenza verso DatabaseManager, un Singleton che si fa carico di inizializzare il database e gestire la connessione (tramite driver JDBC SQLite). Questo approccio astrae le logiche SQL dal resto dell'applicazione, favorendo la manutenibilità e rispettando il Principio di Singola Responsabilità (SRP).

2.6.5 Diagramma delle classi della logica di analisi della term frequency

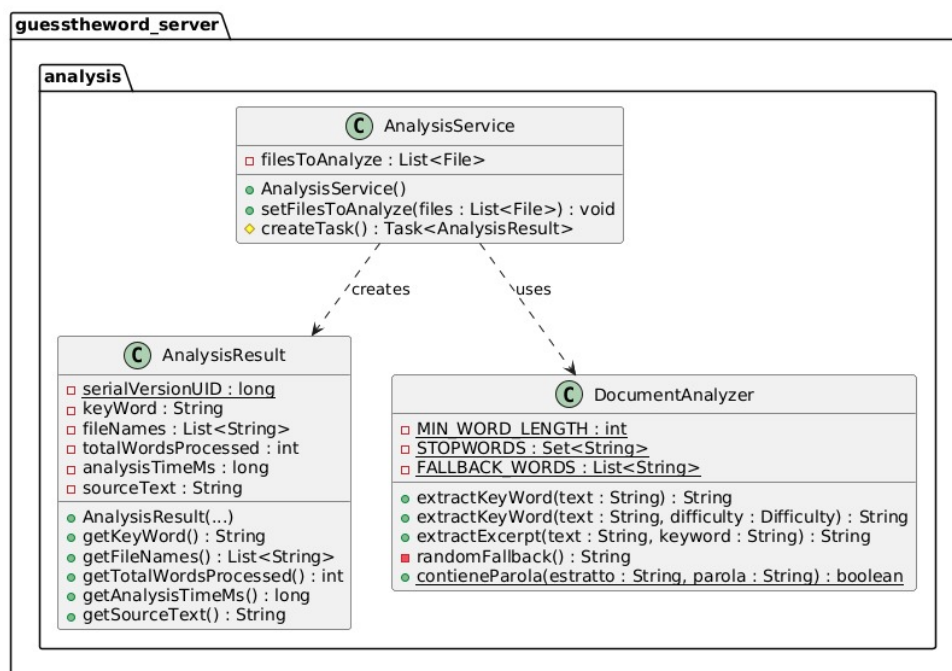


Figura 2.7: Diagramma delle classi della logica di analisi

Questo diagramma mostra il sottosistema del Server adibito all'analisi dei documenti testuali caricati dall'amministratore, funzionalità essenziale per generare dinamicamente nuove sfide di gioco. Il processo viene coordinato da *AnalysisService*, che incapsula la logica in task eseguiti in background per garantire la fluidità della dashboard amministrativa. Tale servizio sfrutta il *DocumentAnalyzer*, un motore di parsing incaricato di processare i file testuali, filtrare le parole non pertinenti (stop words), valutare la lunghezza delle parole ed estrarre la keyword finale assieme a un suo estratto testuale di contesto. Al termine dell'elaborazione, i dati vengono consolidati all'interno della classe di trasporto *AnalysisResult*, che racchiude tutte le metriche dell'operazione (parole processate, tempo impiegato, testi originari) pronte per essere salvate o mostrate a schermo.

2.7 Scelte di progettazione e design pattern

La progettazione del sistema ha seguito pattern consolidati per favorire l'estensibilità ed il disaccoppiamento dei componenti:

- *Singleton Pattern*: applicato a *DatabaseManager* e *ClientRegistry* per garantire un unico punto di accesso globale a risorse condivise critiche, come il pool di connessioni del database ed il registro dei thread client attivi.
- *Data Access Object (DAO) Pattern*: implementato tramite *UserDAO*, *ChallengeDAO* e *ResultDAO* per astrarre le operazioni di persistenza dei dati, isolando il codice relativo alle query SQL dal motore di gioco e dai servizi applicativi.
- *Connection Factory*: la classe *DatabaseManager* fornisce nuove connessioni tramite il metodo *getConnection()* lasciando l'apertura e la chiusura della risorsa al blocco try-with-resources dei singoli DAO, garantendo la thread-safety in esecuzioni concorrenti.

- *Observer Pattern*: implementato in modo implicito sfruttando le proprietà osservabili di JavaFX per legare gli stati asincroni del servizio di analisi testuale alla barra di avanzamento e alle label di stato dell'interfaccia utente.

Capitolo 3

Implementazione

3.1 Interfaccia grafica del sistema

L'interfaccia utente è stata progettata ponendo particolare attenzione alla separazione tra logica di business e presentazione visiva (pattern MVC), sfruttando la tecnologia FXML accoppiata a fogli di stile CSS per la definizione dell'aspetto grafico. Questa scelta ha consentito di strutturare le schermate in modo modulare e facilitare le transizioni di stato dell'applicazione. Il client si presenta con una finestra a dimensione fissa che esegue lo switch dinamico delle scene caricate da file FXML tramite un gestore di scene dedicato (*SceneManager.java*). Per evitare il blocco del thread grafico principale durante le comunicazioni sincrone con il server, le risposte ricevute sulla socket vengono monitorate in background da un task asincrono che notifica il controllore della vista attiva tramite l'aggiornamento di proprietà osservabili, assicurando la fluidità dell'esperienza d'uso dell'utente. Di seguito sono descritte le schermate del sistema con i relativi dettagli implementativi.

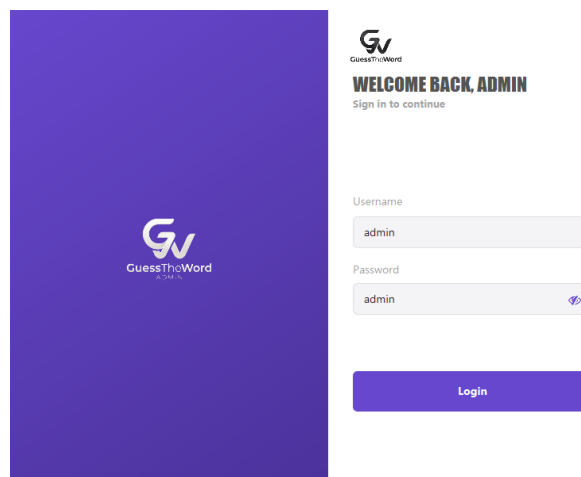


Figura 3.1: Schermata di login admin

La schermata di login per l'amministratore (Figura 3.1) consente l'accesso alle funzionalità di controllo del server previa immissione delle credenziali predefinite. I campi di input prevedono meccanismi di validazione volti a verificare la completezza dei dati prima dell'invio al modulo di autenticazione integrato (credenziali di default: admin / admin).

The screenshot displays the 'Dashboard Administrator' interface. At the top, there are two tabs: 'Classifica Leaderboard' and 'Analisi Frequenza Parole', with the latter being the active tab. Below the tabs, a header section contains the title 'CONSOLE DI AMMINISTRAZIONE' and a description: 'Carica documenti testuali, effettua l'analisi delle frequenze e gestisci la persistenza serializzata.' The main content area is divided into two columns. The left column, titled 'Analisi Frequenza Parole', contains a button 'Selezione Documenti (.txt)' above a file selection area labeled 'File Selezionati:'. This area shows a list of files, with 'fantasy.txt' selected. Below the file list is a button 'Avvia Analisi delle Parole'. The right column, titled 'Persistenza Risultati', contains a description: 'Consente di salvare su disco l'analisi effettuata per non ripeterla ad ogni avvio.' Below this text are two buttons: 'Salva Risultati di Analisi' and 'Carica Risultati Precedenti'. At the bottom of the interface, a status message reads 'Analisi completata con successo.'

Il pannello di amministrazione (Figura 3.2) permette il caricamento di documenti in formato testuale (.txt) da analizzare per l'estrazione delle parole chiave. L'estrazione e il calcolo statistico delle frequenze avvengono in modo asincrono, notificando l'esito tramite messaggi popup operativi. Per ottimizzare le prestazioni della CPU ed evitare elaborazioni ridondanti del medesimo documento, l'interfaccia supporta il salvataggio locale dei risultati in formato binario (.ser) e il loro successivo caricamento rapido.

[illegible]

La vista dello storico generale (Figura 3.3) consente all'amministratore di consultare lo stato delle partite che si sono concluse con la vittoria di uno dei giocatori. La schermata presenta una classifica tabellare contenente username, partite vinte e tempo medio di risposta in millisecondi, ordinata per numero di vittorie e tempo medio di risposta.

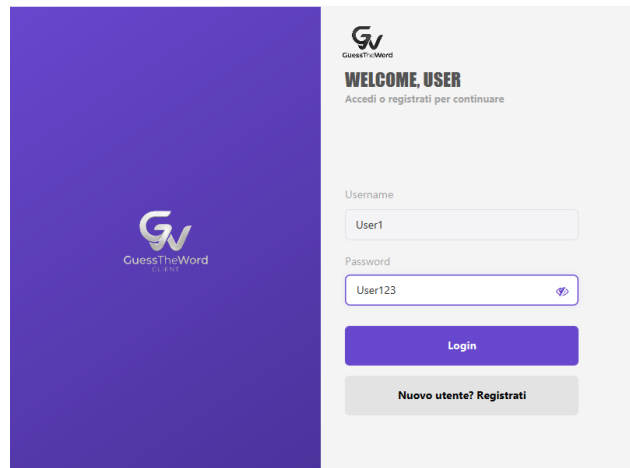


Figura 3.4: Schermata di login

La schermata di login per il giocatore (Figura 3.4) costituisce il punto di ingresso per i giocatori. L'utente immette le proprie credenziali personali (username e password); la password viene verificata dal server confrontandola con l'hash SHA-256 memorizzato nel database. Qualora le credenziali siano errate, l'interfaccia mostra un messaggio informativo di errore, sotto forma di label. Nel caso l'utente non esista nel database, il sistema offre la possibilità di registrarsi (Figura 3.5) passando automaticamente alla schermata di registrazione i valori dei campi immessi.

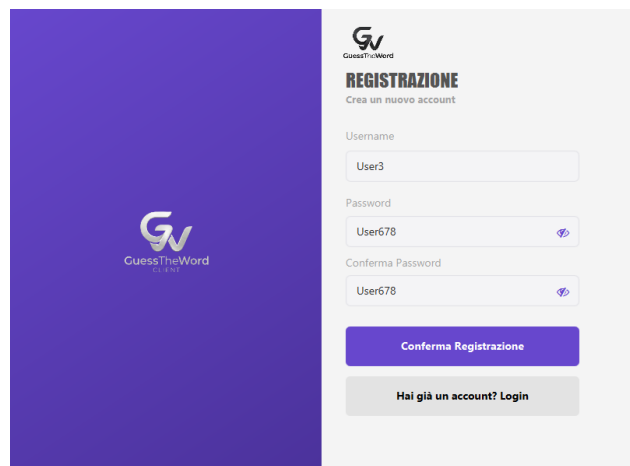


Figura 3.5: Schermata di registrazione utente

La schermata di registrazione (Figura 3.5) permette la creazione di un nuovo profilo utente. L'interfaccia richiede l'inserimento di username, password e della relativa conferma. Il sistema valida la corrispondenza tra i due campi password e la validità sintattica dei campi (ad esempio username di almeno cinque caratteri) prima dell'invio delle credenziali cifrate al server.



Figura 3.6: Lobby di selezione della difficoltà

La lobby principale (Figura 3.6) accoglie l'utente autenticato e permette la selezione della difficoltà di gioco (facile, media o difficile) per l'accoppiamento in partita. Da questa schermata è inoltre possibile accedere al pannello dello storico personale tramite un pulsante dedicato.



Figura 3.7: Lobby di attesa dell'avversario

La stanza d'attesa virtuale (Figura 3.7) viene visualizzata dopo la selezione del livello di difficoltà. Il client rimane in uno stato di attesa attiva mostrando un indicatore di caricamento asincrono, fino a quando il *GameManager* del server non rileva un secondo giocatore connesso con le medesime preferenze di difficoltà, procedendo all'abbinamento e alla inizializzazione della partita.

STORICO PARTITE

Il resoconto di tutte le partite giocate.

Data e Ora	Esito	Difficoltà	Parola Segreta
07/07/2026 16:51	LOSE	EASY	cuore
05/07/2026 20:51	WIN	EASY	aiuto
05/07/2026 20:49	LOSE	EASY	dono
05/07/2026 20:49	WIN	EASY	alta

Nuova Partita

Figura 3.8: Storico delle partite

La schermata dello storico personale del giocatore (Figura 3.8) presenta la lista riepilogativa di tutte le sfide disputate dall'utente loggato. Per ciascuna partita vengono riportati la data, il livello di difficoltà impostato, la parola da indovinare in chiaro e l'esito finale della partita (vittoria, sconfitta o timeout).

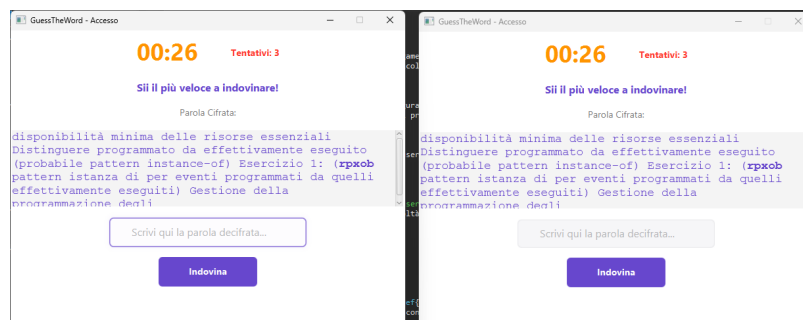


Figura 3.9: Schermata di gioco

La schermata principale di gioco (Figura 3.9) costituisce il nucleo dell'esperienza interattiva. Essa mostra l'estratto testuale con la parola nascosta evidenziata e cifrata tramite il cifrario di Cesare, corredata da un timer di conto alla rovescia di sessanta secondi e dal contatore dei tentativi rimasti. La sottomissione delle risposte genera diversi scenari: se un utente indovina la parola, la sessione si interrompe per entrambi i partecipanti, mostrando la notifica di vittoria ("Hai vinto") al vincitore e di sconfitta ("Hai perso") all'avversario, svelando in chiaro la parola segreta. Qualora scada il timer senza alcuna risposta corretta, la partita termina per timeout mostrando la parola segreta. Se un giocatore esaurisce i tre tentativi prima dell'avversario, il client disabilita il campo di testo e il pulsante di invio, mostrando il messaggio di stato "Tentativi terminati. Attendere la conclusione della partita." e permettendo all'utente di attendere la fine della partita dell'avversario mentre il timer continua a scorrere.

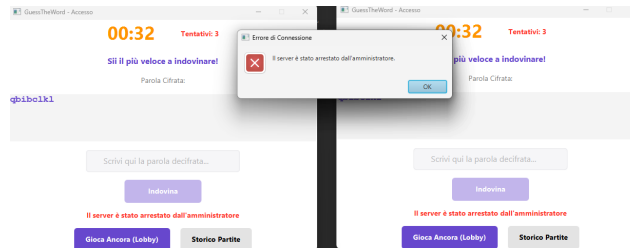


Figura 3.10: Schermata di notifica disconnessione

La schermata di notifica di disconnessione (Figura 3.10) viene mostrata dal client nel momento in cui rileva la chiusura imprevista o intenzionale del server TCP. In tal caso, il client visualizza un popup di errore e interrompe immediatamente qualsiasi operazione in corso, successivamente reindirizza l'utente alla schermata di login. L'accesso al sistema sarà inibito fino al ripristino e il riavvio del server.

3.2 Frammenti di codice rilevanti

In questa sezione vengono riportati i frammenti di codice significativi estratti dalle classi reali del progetto, utili a comprendere i dettagli algoritmici ed implementativi discussi.

3.2.1 Analisi della frequenza dei termini

Il codice seguente, tratto dalla classe *DocumentAnalyzer*, mostra l'uso delle API Stream di Java per estrarre la parola chiave in base alla frequenza e al livello di difficoltà prescelto.

```

1      public String extractKeyWord(String text, Difficulty difficulty)
2      {
3          if (text == null || text.trim().isEmpty()) {
4              return randomFallback();
5          }
6
7          String[] tokens = text.toLowerCase().split("[^a-zA-Zàèìòùáéí
8              óúâêîôûäëïöü]+");
9
10         Map<String, Integer> freqMap = Arrays.stream(tokens)
11             .filter(token -> token.length() >= MIN_WORD_LENGTH)
12             .filter(token -> !STOPWORDS.contains(token))
13             .collect(java.util.stream.Collectors.groupingBy(
14                 token -> token,
15                 java.util.stream.Collectors.summingInt(token -> 1)
16             ));
17
18         if (freqMap.isEmpty()) return randomFallback();
19
20         switch (difficulty) {
21             case EASY:
22                 List<String> easyPool = freqMap.entrySet().stream()
23                     .filter(e -> e.getKey().length() < 6)
24                     .map(Map.Entry::getKey)
25                     .collect(java.util.stream.Collectors.toList());
26                 if (easyPool.isEmpty()) easyPool = new ArrayList<>();
27                 return easyPool.get(new Random().nextInt(easyPool.size()));
28         }
29     }

```

```

27         case HARD:
28             List<String> hardPool = freqMap.entrySet().stream()
29                 .filter(e -> e.getKey().length() >= 9)
30                 .map(Map.Entry::getKey)
31                 .collect(java.util.stream.Collectors.toList());
32             if (hardPool.isEmpty()) hardPool = new ArrayList<>(
33                 freqMap.keySet());
34             return hardPool.get(new Random().nextInt(hardPool.size()
35                 ));
36
37         case MEDIUM:
38         default:
39             List<String> mediumPool = freqMap.entrySet().stream()
40                 .filter(e -> e.getKey().length() >= 6 && e.getKey().
41                     length() <= 8)
42                 .map(Map.Entry::getKey)
43                 .collect(java.util.stream.Collectors.toList());
44             if (mediumPool.isEmpty()) mediumPool = new ArrayList<>(
45                 freqMap.keySet());
46             return mediumPool.get(new Random().nextInt(mediumPool.
47                 size()));
48     }
49 }

```

Listing 3.1: Estrazione della parola chiave per difficoltà in DocumentAnalyzer.java

L'algoritmo di analisi testuale divide in token il testo, rimuovendo i caratteri speciali tramite regex, escludendo le stopwords e i termini più corti. Successivamente, calcola le frequenze assolute dei vocaboli e li suddivide in tre categorie in base alla lunghezza per rispecchiare i livelli di difficoltà: inferiore a 6 caratteri per il livello EASY, tra 6 e 8 caratteri per il livello MEDIUM, e superiore o uguale a 9 caratteri per il livello HARD. Nel caso in cui il sottoinsieme filtrato per la difficoltà selezionata risulti vuoto, il sistema esegue un fallback completo selezionando una parola casuale dal testo estratto.

3.2.2 Cifrario di Cesare e generazione dello shift

La classe *CaesarCipher* si occupa di applicare la cifratura di Cesare ed identificare lo shift casuale in funzione del livello di difficoltà richiesto dal client di gioco.

```

1     public static String encrypt(String word, int shift) {
2         if (word == null || word.isEmpty()) return word;
3         shift = ((shift % 26) + 26) % 26;
4         StringBuilder sb = new StringBuilder();
5         for (char c : word.toCharArray()) {
6             if (Character.isLetter(c)) {
7                 char base = Character.isUpperCase(c) ? 'A' : 'a';
8                 sb.append((char) (base + (c - base + shift) % 26));
9             } else {
10                sb.append(c);
11            }
12        }
13        return sb.toString();
14    }
15
16    public static int randomShift(Difficulty difficulty) {
17        Random random = new Random();
18        switch (difficulty) {
19            case EASY: {

```

```

20         if (random.nextBoolean()) {
21             return 1 + random.nextInt(5);
22         } else {
23             return 21 + random.nextInt(5);
24         }
25     }
26     case MEDIUM: {
27         if (random.nextBoolean()) {
28             return 5 + random.nextInt(4);
29         } else {
30             return 19 + random.nextInt(4);
31         }
32     }
33     case HARD:
34     default: {
35         return 9 + random.nextInt(10);
36     }
37 }
38 }

```

Listing 3.2: Generazione dello shift ed encrypt in CaesarCipher.java

La classe gestisce l'offuscamento delle parole normalizzando matematicamente lo shift modulo 26, preservando la formattazione in maiuscolo o minuscolo dei singoli caratteri. La logica di generazione dello shift casuale adatta la cifratura in base alla difficoltà: nel livello EASY viene selezionato uno shift tra 1 e 5 oppure tra 21 e 25, mantenendo la traslazione vicina agli estremi dell'alfabeto; nel livello MEDIUM lo shift ricade tra 5 e 8 o tra 19 e 22; nel livello HARD viene impostato uno shift centrale compreso tra 9 e 18, massimizzando il disallineamento visivo e rendendo la decifratura manuale estremamente complessa.

3.2.3 Protocollo di comunicazione

La classe *MessageProtocol* implementa i metodi per la composizione e la decomposizione dei pacchetti dati trasmessi sul canale TCP di rete.

```

1     public static String build(String command, String... params) {
2         if (params == null || params.length == 0) return command;
3         return command + "\u001F" + String.join("\u001F", params);
4     }
5
6     public static String[] parse(String message) {
7         return message.split("\u001F");
8     }

```

Listing 3.3: Metodi build e parse in MessageProtocol.java

I metodi di utilità della classe automatizzano la formattazione dei messaggi di rete inserendo il carattere di controllo non stampabile Unit Separator (\u001F) come delimitatore tra il comando principale e la stringa dei parametri associati. La funzione di scomposizione (parse) speculare separa il pacchetto ricevuto ricavandone un array di stringhe. Tale schema previene qualsiasi collisione sintattica con i caratteri speciali della punteggiatura (ad esempio :) o con le lettere accentate inserite dagli utenti.

3.2.4 Gestione della connessione socket

Il server socket viene creato e gestito dalla classe *GameServer*, che accetta le connessioni e le smista ad un pool di thread concorrenti.


```

1      public void startCon() throws IOException {
2          serverSocket = new ServerSocket(port);
3          System.out.println("Server avviato sulla porta " + port);
4          try {
5              while (running) {
6                  Socket clientSocket = serverSocket.accept();
7                  System.out.println("Client connesso: " +
8                      clientSocket.getInetAddress());
9
10                     ClientHandler handler = new ClientHandler(
11                         clientSocket, clientRegistry);
12                     clientRegistry.register(handler);
13                     executorService.submit(handler);
14             }
15         } catch (SocketException e) {
16             if (!running) {
17                 System.out.println("[GameServer] ServerSocket chiuso
18                     correttamente.");
19             } else {
20                 throw e;
21             }
22         }
23     }

```

Listing 3.4: Connessione socket TCP in GameServer.java

Il server di gioco inizializza la socket di ascolto sulla porta configurata ed entra in un ciclo continuo controllato dalla variabile di stato. All'arrivo di una richiesta di connessione tramite la chiamata bloccante `accept()`, viene generata una nuova socket di comunicazione dedicata, la quale viene incapsulata in un'istanza di *ClientHandler*, registrata globalmente e immediatamente sottomessa al pool di esecuzione dell'architettura concorrente per la gestione parallela.

3.2.5 Persistenza tramite pattern DAO

La registrazione di un nuovo profilo utente è demandata al metodo `register` della classe *UserDAO*, che si avvale di prepared statement per scongiurare minacce di iniezione di codice SQL (SQL Injection).

```

1      public boolean register(User user) {
2          String query = "INSERT INTO utenti (username, password,
3              ruolo, data_iscrizione) VALUES (?, ?, ?, ?)";
4          DatabaseManager dbManager = DatabaseManager.getInstance();
5
6          try (Connection conn = dbManager.getConnection();
7              PreparedStatement ps = conn.prepareStatement(query,
8                  Statement.RETURN_GENERATED_KEYS)) {
9
10             ps.setString(1, user.getUsername());
11             ps.setString(2, user.getPassword());
12             ps.setString(3, user.getRuolo());
13             ps.setString(4, user.getDataIscrizione().format(
14                 DATE_FORMATTER));
15
16             int affectedRows = ps.executeUpdate();
17             if (affectedRows == 0) {
18                 return false;
19             }
20         }
21     }

```

```

16         }
17
18         try (ResultSet generatedKeys = ps.getGeneratedKeys()) {
19             if (generatedKeys.next()) {
20                 user.setId(generatedKeys.getInt(1));
21             }
22         }
23         return true;
24
25     } catch (SQLException e) {
26         throw new DataAccessException("Errore durante la
27             registrazione dell'utente: " + user.getUsername(), e)
28             ;
29     }
30 }

```

Listing 3.5: Registrazione utente in UserDao.java

Il metodo di registrazione esegue una query d'inserimento precompilata a livello di database recuperando la connessione tramite il manager dedicato. Il modulo richiede esplicitamente la restituzione delle chiavi primarie (id) autogenerate (autoincrement) per popolare istantaneamente l'identificativo dell'oggetto utente in memoria. L'adozione del blocco try-with-resources garantisce il rilascio immediato delle risorse esterne della connessione in caso di eccezioni SQL.

3.2.6 Inizializzazione dello schema e gestione delle connessioni

Il *DatabaseManager* centralizza l'apertura delle connessioni e si occupa della creazione e migrazione delle tabelle all'avvio dell'applicazione.

```

1     static {
2         try {
3             Class.forName("org.sqlite.JDBC");
4         } catch (ClassNotFoundException e) {
5             System.err.println("[DB] Driver JDBC SQLite non trovato:
6                 " + e.getMessage());
7         }
8     }
9
10    public Connection getConnection() throws SQLException {
11        Connection conn = DriverManager.getConnection(DB_URL);
12        try (Statement stmt = conn.createStatement()) {
13            stmt.execute("PRAGMA foreign_keys = ON;");
14        }
15        return conn;
16    }

```

Listing 3.6: Gestione connessioni JDBC in DatabaseManager.java

La classe centrale per l'interazione JDBC carica staticamente il driver SQLite all'avvio della classe (tramite blocco statico) per ottimizzare le prestazioni a runtime ed evitare caricamenti ridondanti ad ogni connessione, e restituisce una connessione attiva verso il file database locale del server. Prima di rilasciare la connessione ai moduli richiedenti, viene eseguito il comando pragma che impone l'attivazione e la verifica rigorosa dei vincoli di integrità referenziale.

3.2.7 Caching serializzato dell'analisi delle parole

L'interfaccia di amministrazione del server implementa il salvataggio dei dati elaborati in un file cache per evitare il ripetersi di lunghe elaborazioni sui medesimi testi.

```
1      @FXML
2      private void handleSaveResults(ActionEvent event) {
3          if (lastAnalysisResult == null) {
4              showAlert(Alert.AlertType.WARNING, "Attenzione", "Nessun
5                  dato da salvare",
6                  "Nessun risultato disponibile per il salvataggio.");
7              return;
8          }
9          FileChooser fileChooser = new FileChooser();
10         fileChooser.setTitle("Salva cache analisi");
11         fileChooser.getExtensionFilters().add(new FileChooser.
12             ExtensionFilter("File Cache (*.ser)", "*.ser"));
13         fileChooser.setInitialFileName("cache_analisi.ser");
14
15         Stage stage = (Stage) saveResultsBtn.getScene().getWindow();
16         File file = fileChooser.showSaveDialog(stage);
17
18         if (file != null) {
19             try (ObjectOutputStream oos = new ObjectOutputStream(new
20                 FileOutputStream(file))) {
21                 oos.writeObject(lastAnalysisResult);
22                 cacheStatusLabel.setText("Cache: Salvata (" + file.
23                     getName() + ")");
24                 showAlert(Alert.AlertType.INFORMATION, "Salvataggio
25                     Cache", "Salvataggio completato!",
26                     "La cache dei risultati è stata salvata con successo
27                     in:\n" + file.getName());
28                 System.out.println("[Dashboard] Cache salvata con
29                     successo in " + file.getAbsolutePath());
30             } catch (Exception e) {
31                 cacheStatusLabel.setText("Cache: Errore di
32                     salvataggio");
33                 showAlert(Alert.AlertType.ERROR, "Errore di
34                     Salvataggio", "Impossibile salvare la cache", e.
35                     getMessage());
36                 e.printStackTrace();
37             }
38         }
39     }
```

Listing 3.7: Salvataggio della cache serializzata in AdminDashboardViewController.java

Il controller della dashboard amministrativa cattura l'azione di salvataggio dell'amministratore aprendo una finestra di dialogo per definire il nome del file cache e salvarlo nel filesystem. Mediante un flusso di scrittura di oggetti (*ObjectOutputStream*), l'istanza contenente i risultati dell'analisi testuale viene serializzata e salvata in formato binario (.ser), mostrando un alert di conferma ed emettendo un log sulla console. La controparte di lettura ripristina lo stato completo dell'analisi tramite deserializzazione asincrona, ripopolando l'interfaccia senza rieseguire calcoli intensivi sulla CPU.

Capitolo 4

Conclusioni e sviluppi futuri

4.1 Risultati conseguiti

Con la realizzazione del progetto GuessTheWord abbiamo sviluppato ed integrato con successo un'applicazione cooperativa e competitiva basata su un'architettura client-server multithread. L'interfaccia grafica sviluppata in JavaFX ha risposto ai criteri di usabilità e fluidità grafica grazie all'esecuzione non bloccante della logica di rete e dei calcoli di analisi su thread di background, mentre la persistenza strutturata tramite SQLite e l'implementazione del pattern DAO hanno garantito la stabilità, l'integrità referenziale e la tracciabilità delle performance degli utenti registrati.

4.2 Limitazioni riscontrate

Durante la fase di collaudo del sistema abbiamo identificato alcune limitazioni intrinseche che meritano di essere evidenziate:

- *Concorrenza del database SQLite*: l'adozione di un motore database embedded memorizzato su un singolo file comporta limitazioni prestazionali nel caso di accessi in scrittura simultanei ad alta frequenza, in quanto SQLite blocca l'intero file durante le transazioni di update e insert, potendo causare eccezioni di tipo database locked.
- *Sicurezza del protocollo di comunicazione*: i messaggi scambiati sui socket TCP viaggiano in chiaro sulla rete. Sebbene le credenziali degli utenti vengano verificate tramite hash SHA-256 impedendo il transito delle password in chiaro, l'assenza di uno strato di cifratura del canale (come SSL/TLS) espone il sistema ad attacchi di intercettazione ed all'eventuale alterazione del traffico di gioco.
- *Architettura a Thread Bloccanti per Client*: il server alloca un thread nativo del sistema operativo per ciascun client connesso tramite il pool fisso di thread. Questa soluzione limita la scalabilità orizzontale del server a poche centinaia di utenti simultanei prima di saturare le risorse hardware della CPU.

4.3 Sviluppi futuri

Per superare i limiti riscontrati ed estendere le funzionalità del sistema, si prospettano i seguenti interventi migliorativi:

- *Integrazione di Virtual Thread (Project Loom)*: l'introduzione dei thread virtuali introdotti in Java 21, con conseguente migrazione del progetto a tale versione del JDK, permetterebbe al server di gestire migliaia di connessioni concorrenti allocando thread leg-

geri gestiti direttamente dalla JVM, riducendo drasticamente il consumo di memoria e migliorando la scalabilità complessiva del sistema.

- *Migrazione a DBMS Centralizzato*: il passaggio da SQLite ad un motore database relazionale di rete centralizzato, come PostgreSQL, risolverebbe i colli di bottiglia legati ai lock di scrittura concorrente sulle tabelle dei risultati e delle sfide.
- *Cifratura del Canale SSL/TLS*: l'integrazione di socket SSL (tramite SSLSocketFactory e SSLServerSocketFactory) garantirebbe la riservatezza, l'autenticità e l'integrità di tutte le comunicazioni di rete tra client e server.